MACHINE LEARNING WITH SCIKIT-LEARN

Data Understanding & Cleaning

02

Real data is messy. Learn to look at it properly, find what's broken, and fix it without lying to your model.

Master's Degree Program · Hands-on with Google Colab

AGENDA

Today's agenda

01 **Why cleaning matters**
garbage in, garbage out

02 **Anatomy of a dataset**
features X, target y, dtypes

03 **EDA — look before you model**
distributions · target · groups

04 **Missing data**
find it, understand it, impute it

05 **Duplicates**
quiet score-inflators

06 **Outliers**
detect, investigate, then decide

07 **Inconsistent categories**
"BKK" vs "Bangkok"

08 **The cleaning checklist**
your reusable routine

THE 80% TRUTH

Data scientists spend most time here

~80%

of a data scientist's time goes to preparing & cleaning data — not modeling

- **Why we can't skip it:**
- Models don't “fix” dirty data — they learn it
- Median-imputing a typo (age = 800) shifts everything
- Leaked duplicates = a test set your model already memorized
- **Session goal: a repeatable routine you'll run on every dataset — including your final project.**

WHERE WE ARE

This session = steps 2–3 of the workflow

1
1 · Frame
done in S01

→

2
2 · Collect & EDA
today — look & question

→

3
3 · Prepare
today: clean · next: transform

→

4
4–8 · Train → share
sessions 04–08

Understand (EDA)

What is each column? What's missing? What's weird? How does the target behave? EDA is asking the data questions — before it asks you for a model.

Clean

Fix missing values, drop/reconcile duplicates, handle outliers, normalize categories. Cleaning must happen BEFORE the train/test split (except… we'll see why in S03).

RUNNING EXAMPLE

The Titanic dataset — our lab rat this session

y: survived	pclass	sex	age	sibsp	fare
0	3	male	22.0	1	7.25
1	1	female	38.0	1	71.28
1	3	female	26.0	0	7.93

891 passengers · 8 columns · target: survived (0/1) — did they live?

```
import pandas as pd, seaborn as sns
df = sns.load_dataset("titanic")[["survived","pclass","sex","age",
                                  "sibsp","parch","fare","embarked"]]
```

- **X = features**
- pclass, sex, age, sibsp, parch, fare, embarked
- **y = target: survived**
- Task: binary classification — predict survival from passenger info

— First studied in 1912… still teaching in 2026.

COLUMN TYPES

Know your dtypes — they steer every decision

Numerical

int64 / float64

age, fare, sibsp, parch

→ mean/median, histograms, scaling, outliers

Categorical

object / category

sex, embarked

→ value_counts(), encoding (S03): one-hot, ordinal

Ordinal (in between)

ordered categories

pclass 1st < 2nd < 3rd

→ keep the order (map 1,2,3), don't one-hot blindly

df.dtypes is the first thing I print — half of ML bugs on tabular data are dtype surprises ("age" stored as object because of "42 y/o").

EDA STEP 1

The first five minutes with any dataset

```
df.shape                # (891, 8)  rows, columns
df.head(10)             # eyeball real values
df.info()               # dtypes + missing per column
df.describe()           # numeric summary
df["sex"].value_counts()        # category sanity
df["embarked"].value_counts(dropna=False)
```

```
df["embarked"].value_counts(dropna=False)

embarked
S      644
C      168
Q       77
NaN      2   <- count the missing too!
```

- **What you're hunting:**
- missing values & their pattern
- impossible values (negative age?)
- weird categories / typos
- surprising ranges (fare = 0?)
- **Run all five on EVERY dataset you ever meet. No exceptions.**

EDA STEP 1

df.info() — read it like a detective

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 8 columns):
 #   Column    Non-Null  Dtype
---  ------    --------  -----
 0   survived  891       int64
 1   pclass    891       int64
 2   sex       891       object
 3   age       714  <<<   float64
 4   sibsp     891       int64
 5   parch     891       int64
 6   fare      891       float64
 7   embarked  889  <<<   object
```

- **age: 714 / 891 → 177 missing (19.9%)**
- embarked: 2 missing — tiny, easy
- **Questions to ask next:**
- Which passengers lack age? Random — or 3rd class? (missingness can be a signal)
- How will we fill them? → coming in 4 slides

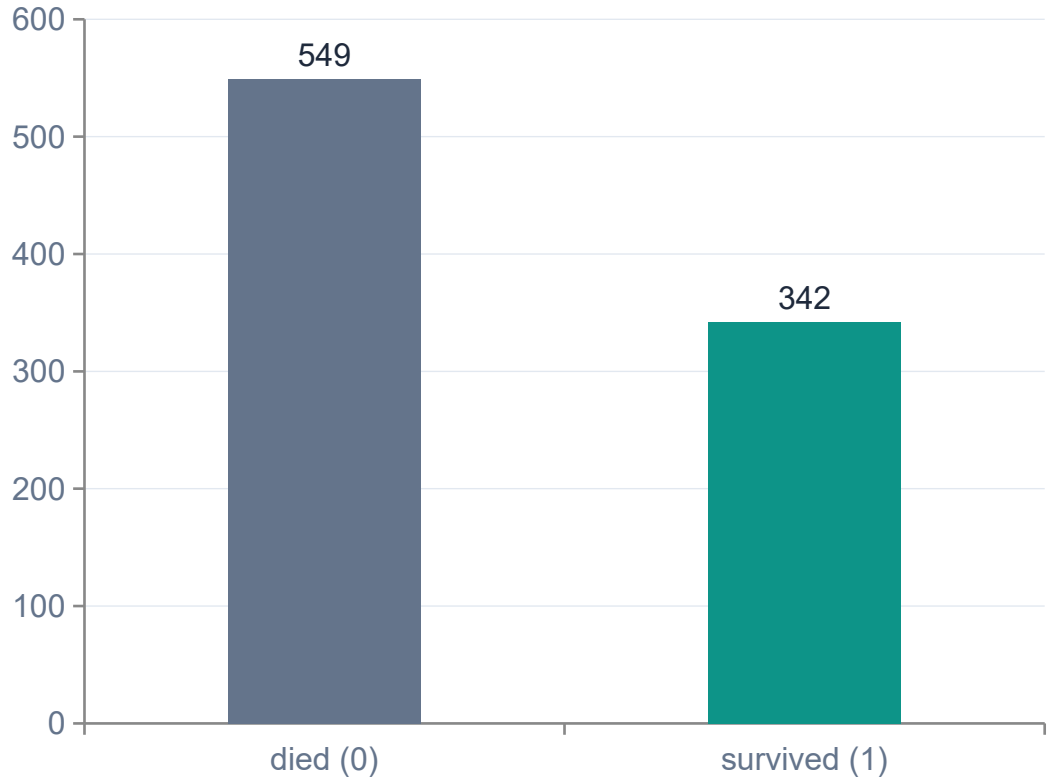
EDA STEP 1

df.describe() — sanity check in numbers

	age	fare	sibsp	parch
min	0.42	0.00	0	0
50%	28.0	14.45	0	0
max	80.0	512.33	8	6
mean	29.7	32.20	0.52	0.38

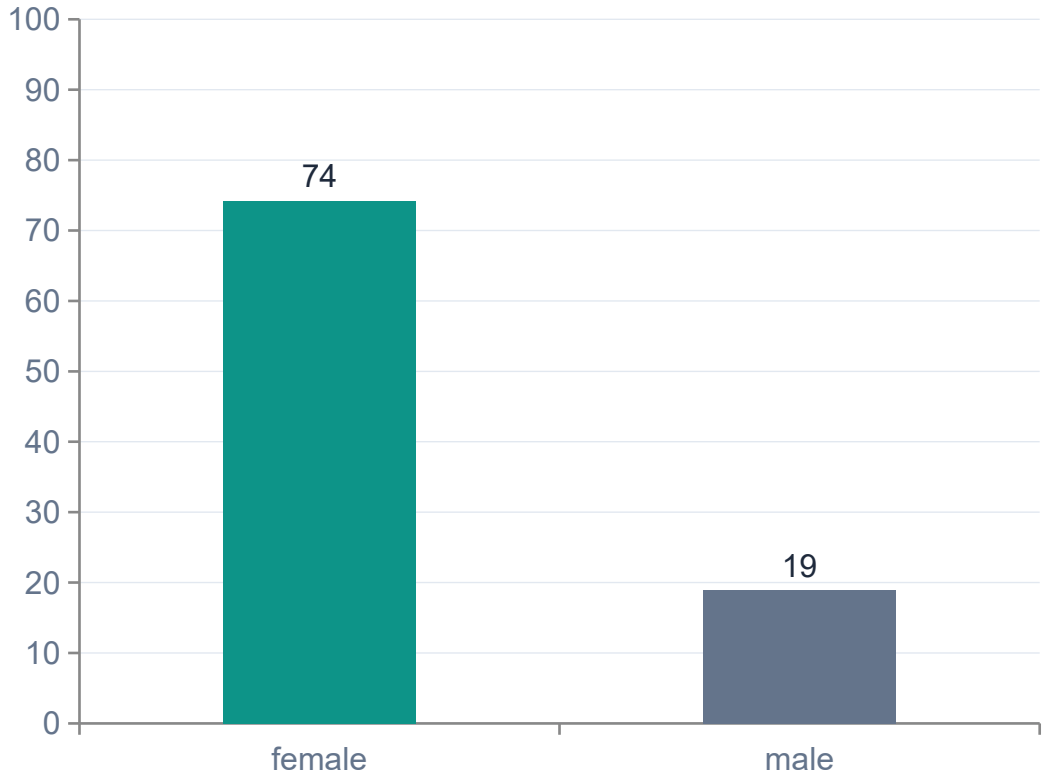
- **age min 0.42 → infants on board. Real, keep.**
- **fare min 0.00 → crew / complimentary tickets?**
- investigate: df[df.fare == 0] — 15 rows
- **fare max 512 vs median 14.45 → wildly skewed**
- the mean (32.2) lies; use the median
- **max is never "wrong" until you check — outliers are suspects, not criminals**

Always start EDA with the target



- **38.4% survived — the base rate**
- A model that always says “died” scores 61.6% accuracy
- **That's your DummyClassifier — the number to beat (S04)**
- This is also where you spot class imbalance
- Here 62/38 is mild; fraud detection can be 99.4/0.6 (!)

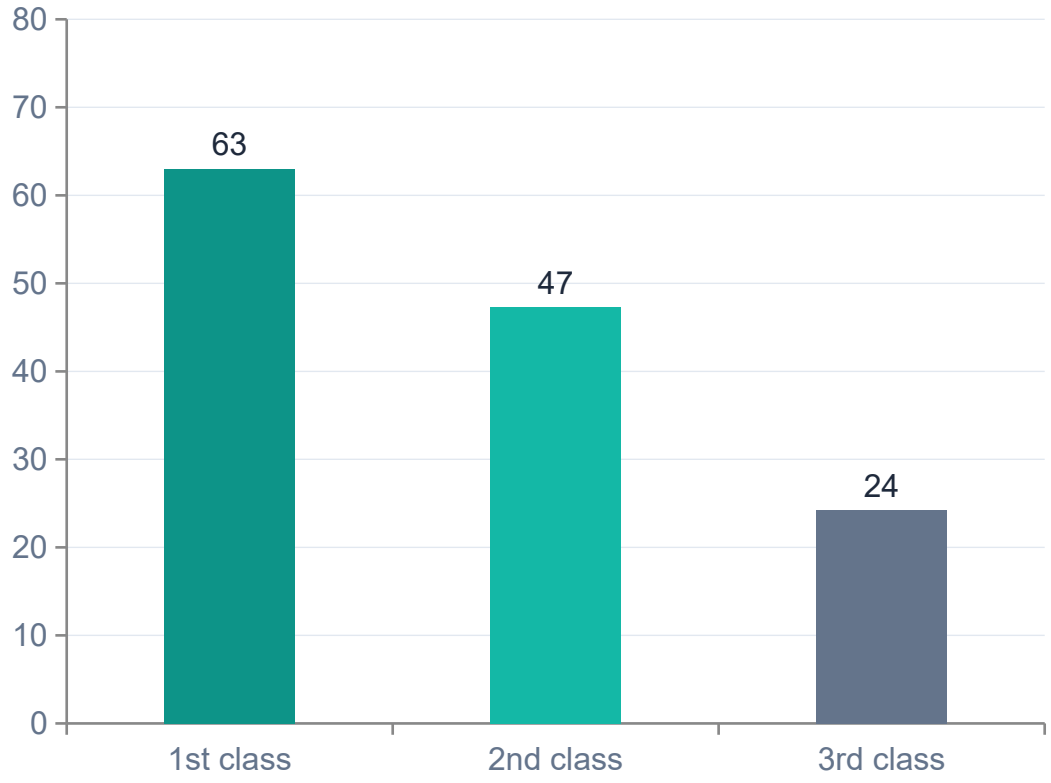
“Women and children first” — is it in the data?




```
df.groupby("sex")["survived"].mean()
```

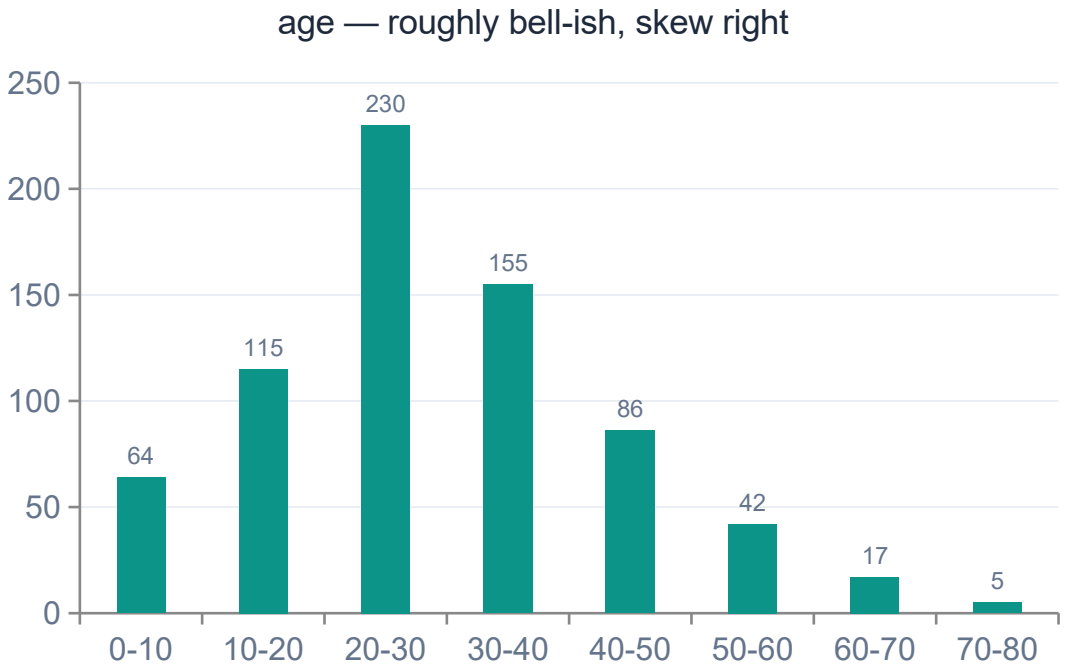
- **74% vs 19% — sex alone is a powerful feature**
- groupby().mean() is the workhorse of EDA
- You just did “hypothesis-driven EDA”: history said X, data confirms/refutes

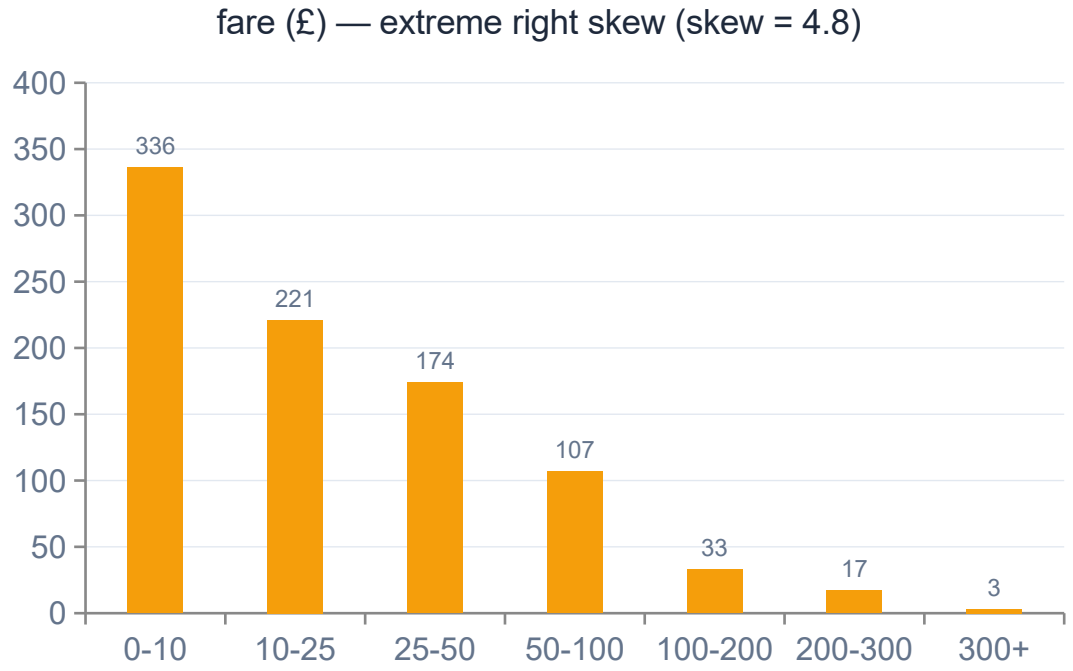
Money bought lifeboats



- **1st class: 63% · 3rd class: 24%**
- pclass is ordinal — 1 > 2 > 3 in privilege
- and it correlates with fare…
- **Question for the lab: female + 3rd class — which effect wins?**
- Hint: df.pivot_table(index="pclass", columns="sex", values="survived")

Know your shapes: age vs fare





Skewed columns → prefer MEDIAN over mean, and consider a log transform in S03. Distribution shape decides your tools.

Correlation — a quick map, not the territory

feature	corr with survived
pclass	−0.34
fare	+0.26
age	−0.08
sibsp	−0.04
parch	+0.08

```
sns.heatmap(df.corr(numeric_only=True), annot=True)
```

- **pclass −0.34: higher class (1) → survived more**
- **but sex (74% vs 19%) barely shows up here!**
- correlation only sees LINEAR numeric relations
- sex is categorical — compare group means instead
- **corr is a metal detector: it finds where to dig, not what's buried**

Not all missing is equal

Missing completely at random

Typos, sensor glitch — no pattern. Safe to impute simply.

Missing with a reason

3rd-class records were sloppier → age missing correlates with pclass. The GAP ITSELF is information.

Missing by design

deck is NaN for 77% — mostly 3rd class (no cabin assigned). This is a story about wealth, not lost data.

Practical rule: plot missingness against the target & features (sns.heatmap of isna()). If it relates → make “was_missing” a feature.

Two lazy fixes and their price

```
df.dropna()           # 💥 loses 179 rows = 20% of the data
df["age"].fillna(0)   # 💥 0 is a baby — 177 fake infants
```

- **Sometimes dropna IS right:**
- when 2/891 rows are affected (embarked)
- when the whole row is junk

dropna()

Simple, honest — but here it throws away 1st-class passengers' rows too, and biases your data toward whoever kept good records.

fillna(0)

Silent lie: age 0.0 = infant. Models trust your numbers. Every imputation invents data — make it a PLAUSIBLE invention.

Impute — fill with a defensible guess

column type	strategy	why
numeric, skewed	median	robust to outliers
numeric, symmetric	mean	fine when no outliers
categorical	most_frequent	the mode category
rare & important	constant	fill "Unknown" → its own category

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median")
df[["age", "fare"]] = imp.fit_transform(df[["age", "fare"]])
# (in S03 we'll do this INSIDE a Pipeline — no leakage)
```

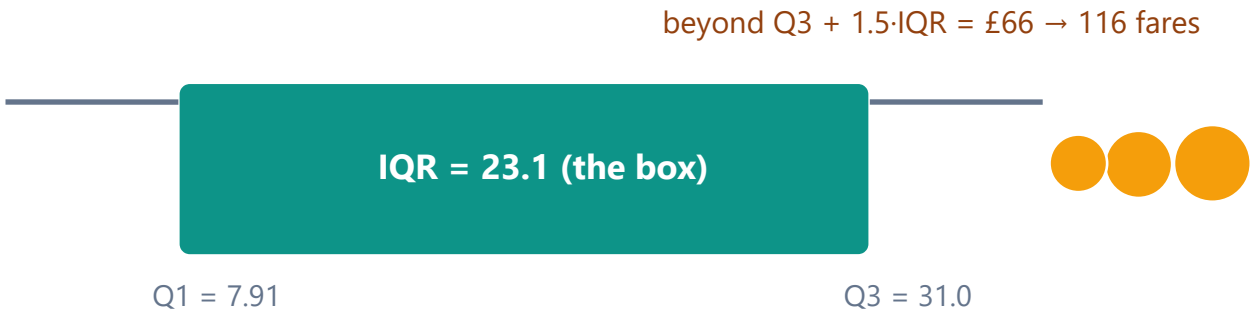
- **Why not pandas fillna?**
- fillna is fine for quick EDA…
- **but sklearn's imputer remembers the training median and applies it to future data — reproducible, leak-safe**

111 identical rows — innocent or dangerous?

```
df.duplicated().sum()      # 111
df[df.duplicated(keep=False)].sort_values("fare").head(6)
```

- **Here: plausible — many passengers share all 8 coarse values**
- Not copied-pasted accidents
- **Danger case: duplicates across train/test split**
- the model gets quizzed on exact rows it studied → inflated score
- **Rule: check duplicates ALWAYS; drop when suspicious (df.drop_duplicates())**

Finding outliers — the IQR fence




```
q1, q3 = df["fare"].quantile([0.25, 0.75])
iqr = q3 - q1
fence_hi = q3 + 1.5 * iqr          # 66.06
outliers = df[df["fare"] > fence_hi]
len(outliers)                       # 116  (age: only 11)
```

- **116 “outliers” in fare?!**
- In skewed data the fence flags the TAIL, not errors
- **Options (in order of preference):**
- keep + use median/robust stats
- log-transform the column (S03)
- cap (winsorize) at a percentile
- delete only with a story why

An outlier is a suspect, not a criminal

The £512.33 ticket is REAL

Three first-class passengers really paid it (the Fortunes, with their maid). Deleting them removes the signal "extreme wealth → survived".

But age = 800 would be fake

A keystroke error. It stretches every mean, every scale, every distance-based model (KNN!). Fix or remove — after checking the raw record.

Before deleting anything: df[df.fare > 300] → look at THOSE rows. Real people? Keep. Broken record? Kill. Decide with evidence, not vibes.

CATEGORIES

"BKK" vs "Bangkok" — same, different, or new?

```
df["city"].value_counts()

Bangkok     412
BKK         173   <<< same city, two spellings
bangkok      42   <<< case
Khonkaen    118
ขอนแก่น      19   <<< Thai/English mixed
```

```
city_map = {"BKK": "Bangkok", "bangkok": "Bangkok", "ขอนแก่น":
"Khonkaen"}
df["city"] = df["city"].replace(city_map)
```

- **value_counts() is your weapon**
- Run it on EVERY object column
- Watch for: case, whitespace (" Bangkok"), synonyms, typos, mixed languages
- **Left alone, one city becomes 4 “different” categories → thinner data, dumber model**
- Also: strip whitespace: df["city"].str.strip()

SUMMARY

The cleaning checklist — run on every dataset

1
shape & head
know what a row IS

→

2
info()
dtypes + missing

→

3
describe()
sanity ranges

→

4
target EDA
base rate + imbalance

1
value_counts()
category sanity

→

2
impute
median / mode / Unknown

→

3
duplicates
check → decide

→

4
outliers
investigate → decide

Print it, tape it to your monitor. This exact list is Lab 02 — and Session 08's project.

NEXT SESSION

Clean ≠ model-ready

sex is still the word “female”. fare spans £0–£512. Models eat NUMBERS on comparable scales.

Encode

female/male → 1/0 or one-hot columns. embarked S/C/Q → 3 columns.

Scale

age 0–80 and fare 0–512 → comparable ranges. KNN & friends will thank you.

Engineer

family = sibsp + parch + 1 · fare_log · child = age < 12

Session 03 — and the single most important sklearn idea: Pipelines (no leakage, ever).

LAB 02 · DO IT IN GOOGLE COLAB

ทำความรู้จักกับ **Titanic — Clean it like a pro**

YOUR MISSION (โจทย์ภาษาไทยใน Colab)

- โหลด titanic จาก seaborn แล้วตรวจ shape / head / info / describe
- หา column ที่มี missing — เขียนว่าจะ impute ด้วยอะไร เพราะอะไร
- สำรวจ target: อัตรารอดชีวิตรวม แยกตาม sex และ pclass
- ตรวจ duplicates และ outliers ของ fare (IQR) แล้วตัดสินใจว่าจะทำอะไร
- โจทย์ชวนคิด: ผู้หญิงชั้น 3 กับผู้ชายชั้น 1 ใครรอดมากกว่ากัน? (pivot_table)

Learning goal

Run the full EDA + cleaning checklist on the Titanic dataset and justify every decision you make.

Vibe coding corner

ลองถาม AI: “ช่วยเขียนโค้ดสำรวจ missingness — วาด heatmap ของ df.isna() และเทียบอัตรา missing ของ age แยกตาม pclass” แล้วตีความผลเองว่า missing เป็นแบบสุ่มหรือมีรูปแบบ

KEY TAKEAWAYS

1. EDA first, models later: head / info / describe / value_counts on every dataset.
2. Start with the target — 38.4% survived is the baseline your model must beat.
3. Impute deliberately: median for skewed numbers, mode for categories, never blanket dropna().
4. Duplicates can quietly inflate your score; outliers are suspects — investigate before deleting.
5. Clean data ≠ ready data: encoding & scaling come next, inside Pipelines.

NEXT

Session 03 — Feature Engineering & Pipelines — numbers in, no leakage